



UNICA

UNIVERSITÀ  
DEGLI STUDI  
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

# Sviluppo Prime API

Angelo Sotgiu

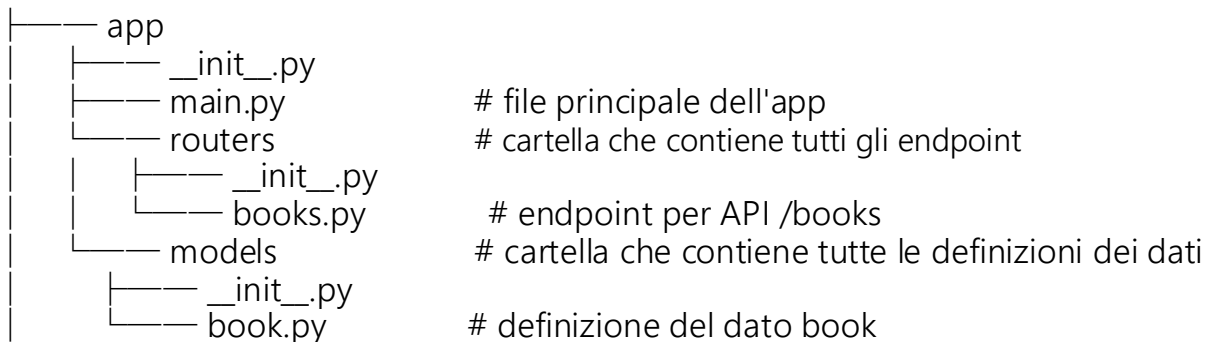
[angelo.sotgiu@unica.it](mailto:angelo.sotgiu@unica.it)

# Riprendiamo l'esempio della biblioteca...

- Abbiamo bisogno di API per:
  - Ottenere la lista di libri -> GET /books
  - Ottenere il dettaglio di un singolo libro -> GET /books/{id}
  - Inserire una recensione per il libro scelto -> POST /books/{id}
- Per ogni libro, in questo caso d'uso specifico, ci serviranno gli attributi:
  - ID (intero)
  - Titolo (stringa)
  - Autore (stringa)
  - Recensione (intero tra 1 e 5)
- Possiamo implementare una classe che definisce la struttura dati "Book"
  - Useremo Pydantic, che farà automaticamente validazione dei dati contenuti nella struttura

# Organizzazione del codice

- Prima di scrivere il codice, è meglio già pensare a come organizzarlo in maniera modulare, evitando di scrivere tutto dentro un file.
  - In questo modo sarà più facile capire, mantenere, testare e riutilizzare il codice.
- Non c'è uno standard unico, e le caratteristiche del progetto specifico possono orientare l'organizzazione.
- Usiamo quella consigliata da FastAPI, con qualche aggiunta:



# Struttura dati per libri

- Definiamo la classe Book, ereditando dalla classe BaseModel di pydantic. È sufficiente definire gli attributi e il tipo di dato atteso. Potremmo anche definire valori di default e validazione avanzata.

```
from pydantic import BaseModel, Field
from typing import Annotated
```

```
class Book(BaseModel):
    id: int
    title: str
    author: str
    review: Annotated[int, Field(ge=1, le=5)] = None
```

```
components:
  schemas:
    Book:
      type: object
      properties:
        id:
          type: integer
          example: 1
        title:
          type: string
          example: "Il nome della rosa"
        author:
          type: string
          example: "Umberto Eco"
        review:
          type: integer
          minimum: 1
          maximum: 5
          example: 5
```



# Schema di esempio

- Possiamo anche aggiungere esempi che verranno mostrati nella documentazione, sia per l'intero schema...

```
class Book(BaseModel):  
    ...  
  
    model_config = {  
        "json_schema_extra": {  
            "examples": [  
                {  
                    "id": 1,  
                    "title": "Il nome della Rosa",  
                    "author": "Umberto Eco",  
                    "review": 5  
                }  
            ]  
        }  
    }
```

# Schema di esempio

- ...che per i singoli campi

```
from pydantic import BaseModel, Field
from typing import Annotated
```

```
class Book(BaseModel):
    id: Field(int, examples=[1])
    title: Field(str, examples=["Il nome della Rosa"])
    author: Field(str, examples=["Umberto Eco"])
    review: Annotated[int, Field(ge=1, le=5, examples=[5])] = None
```

# Creazione di dati fittizi

- Simuliamo adesso un database che contiene già un elenco di libri.  
Creiamo un package **data** con un file `books.py` al suo interno, contenente un dizionario:

```
from app.models.book import Book
```

```
books = {  
    0: Book(id=0, title="libro zero", author="autore zero", review=3),  
    1: Book(id=1, title="libro uno", author="autore uno", review=5),  
    2: Book(id=1, title="libro due", author="autore due", review=1),  
}
```

- Attenzione: se l'import non funziona, è perché bisogna settare il path da cui l'interprete Python va a leggere i file. Potete farlo in due modi:
  - dall'IDE, impostate come "Sources root" la cartella `app`
  - da terminale, digitate

```
export PYTHONPATH=$PYTHONPATH:<path_cartella_app>
```

# API /books

- Possiamo finalmente implementare la prima API! Ricordiamoci anche la documentazione!!!
- Al posto di farlo nel main.py, scriveremo il codice nel file **routers/books.py**

```
from fastapi import APIRouter # usiamo questa classe al posto di FastAPI
from app.models.book import Book
from app.data.books import books
```

```
router = APIRouter(prefix="/books") # tutti gli endpoint del router partiranno da /books
```

```
@router.get("/")
def get_all_books() -> list[Book]:
    """Returns the list of available books."""
    return list(books.values())
```



# File principale dell'app

- Nel main, dobbiamo come sempre inizializzare l'applicazione web, includendo stavolta i router

```
from fastapi import FastAPI
from routers import books
```

```
app = FastAPI()
app.include_router(books.router, tags=["books"])
```

- Il campo "tags" è facoltativo, ma è utile per la documentazione:  
diamoci uno sguardo -> <http://127.0.0.1:8000/docs>
- NB: per qualsiasi errore, il server risponde di default con status 500 e messaggio "Internal Server Error"



## API /books/{id}

- Definiamo l'endpoint per la seconda API sfruttando i path parameter

```
@router.get("/{id}")
def get_book_by_id(id: int) -> Book:
    """Returns the book with the given ID."""
    return books[id]
```

- Posso anche aggiungere una descrizione per i parametri in questo modo:

```
from fastapi import APIRouter, Path
from typing import Annotated
```

```
@router.get("/{id}")
def get_book_by_id(
    id: Annotated[int, Path(description="The ID of the book to get")]
) -> Book:
    """Returns the book with the given ID."""
    return books[id]
```

## API /books/{id}

- Dobbiamo ora gestire il caso in cui non esista un libro con l'id richiesto nel path:

```
from fastapi import APIRouter, Path, HTTPException
```

```
@router.get("/{id}")
def get_book_by_id(
    id: Annotated[int, Path(description="The ID of the book to get")]
) -> Book:
    """Returns the book with the given ID."""
    try:
        return books[id]
    except KeyError:
        raise HTTPException(status_code=404, detail="Book not found")
```

- Nella documentazione automatica purtroppo non viene incluso lo status 404

## API /books/{id}/review

- Definiamo la struttura dati che accettiamo tramite un BaseModel Pydantic, dentro un nuovo file models/review.py

```
from pydantic import BaseModel, Field
from typing import Annotated
```

```
class Review(BaseModel):
    review: Annotated[int, Field(ge=1, le=5, examples=[5])]
```



## API /books/{id}/review

- Creiamo la funzione di endpoint

```
@router.post("/{id}/review")
def add_review(
    id: Annotated[int, Path(description="The ID of the book to which add
    the review")],
    review: Review
):
    """Adds a review to the book with the given ID."""
    try:
        books[id].review = review.review
        return "Review successfully added"
    except KeyError:
        raise HTTPException(status_code=404, detail="Book not found")
```



## API /books - POST

- Possiamo adesso estendere l'applicazione in modo da prendere in input altri libri, accettando richieste POST sull'endpoint /books
- Per adesso scegliamo di non accettare inserimenti di libri il cui ID esiste già: per convenzione di usa il codice errore 403

```
@router.post("/")
def add_book(book: Book):
    """Adds a new book."""
    if book.id in books:
        raise HTTPException(status_code=403, detail="Book ID already exists")
    books[book.id] = book
    return "Book successfully added"
```



## API /books/{id} - PUT

- Il metodo PUT viene usato per rimpiazzare completamente una risorsa

```
@router.put("/{id}")
def update_book(
    id: Annotated[int, Path(description="The ID of the book to update")],
    book: Book
):
    """Updates the book with the given ID."""
    if not id in books:
        raise HTTPException(status_code=404, detail="Book not found")
    books[id] = book
    return "Book successfully updated"
```

- Analogamente, il metodo PATCH può essere usato per rimpiazzare solo qualche campo della risorsa

## API /books e /books/{id} - DELETE

- Possiamo anche definire dei metodi per eliminare un libro archiviato, o tutti quanti

```
@router.delete("/")
def delete_all_books():
    """Deletes all the stored books."""
    books.clear()
    return "All books successfully deleted"
```

```
@router.delete("/{id}")
def delete_book(
    id: Annotated[int, Path(description="The ID of the book to delete")]
):
    """Deletes the book with the given ID."""
    try:
        del books[id]
        return "Book successfully deleted"
    except KeyError:
        raise HTTPException(status_code=404, detail="Book not found")
```



# Query Parameters

- Solitamente sono utilizzati per specificare delle opzioni su come la risposta verrà fornita  
Es.: filtraggio, ordinamento, etc.
- Aggiungiamo un parametro query sull'API /books per ordinare libri per valore recensione

```
@router.get("/")
def get_all_books(
    sort: Annotated[bool, Query(description="Sort books by their review")] = False
) -> list[Book]:
    """Returns the list of available books."""
    if sort:
        return sorted(books.values(), key=lambda book: book.review)
    else:
        return list(books.values())
```